

Object-Oriented Programming

1

- Object Creation - a Detailed Analysis
- Assigning a Reference
- Testing Objects For Equality
- Passing References as Arguments
- Method-Call Chaining
- Overloaded Methods
- Constructors
- Overloaded Constructors

Object Creation - a Detailed Analysis

2

- Code fragment:

```
1. Car car;  
2. car = new Car();  
3. car.year = 2008;
```

reference variable declaration

object instantiation

Assign 2008 to car's year instance variable

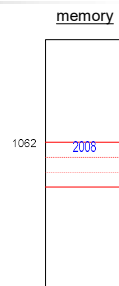
Object Creation - a Detailed Analysis

3

- Code fragment:

```
1. Car car;  
2. car = new Car();  
3. car.year = 2008;
```

car
1062



Assigning a Reference

5

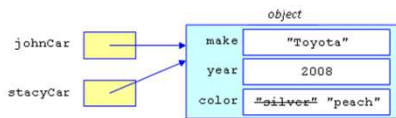
- Suppose you want to create two `Car` objects that are the same except for their color. Your plan is to create the first car, copy the first car to the second car, and then update the second car's `color` instance variable. Will this code accomplish that?

```
Car stacyCar;  
Car johnCar = new Car();  
johnCar.setMake("Toyota");  
johnCar.setYear(2008);  
johnCar.setColor("silver");  
stacyCar = johnCar;  
stacyCar.setColor("peach");
```

Assigning a Reference

5

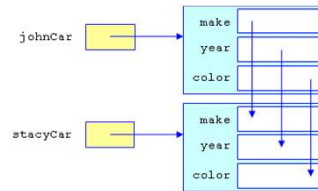
```
Car stacyCar;
Car johnCar = new Car();
johnCar.setMake("Toyota");
johnCar.setYear(2008);
johnCar.setColor("silver");
stacyCar = johnCar;
stacyCar.setColor("peach");
```



Assigning a Reference

7

```
johnCar = new Car();
stacyCar = new Car();
<assign johnCar instance variables to stacyCar instance variables>
```



Assigning a Reference

9

```
public class Car
{
    private String make;
    private int year;
    private String color;
    ...
    public Car makeCopy()
    {
        Car car = new Car();
        car.make = this.make;
        car.year = this.year;
        car.color = this.color;
        return car;
    } // end makeCarCopy
} // end class Car

public static void main(String[] args)
{
    Car johnCar = new Car();
    Car stacyCar;

    johnCar.setMake("Toyota");
    johnCar.setYear(2008);
    johnCar.setColor("silver");
    stacyCar = johnCar.makeCopy();
    stacyCar.setColor("peach");
} // end main
```

Testing Objects for Equality

11

- Using the `==` operator:
 - When comparing two reference variables with `==`, you'd probably expect `==` to return `true` if the data in the two reference variables are the same.

```
Car car1 = new Car();
car1.setColor("red");
Car car2 = new Car();
car2.setColor("red");
if (car1 == car2)
{
    System.out.println("the same");
}
else
{
    System.out.println("different");
}
```

The `car1 == car2` expression returns false. Why?

Testing Objects for Equality

12

- Using the == operator (continued):
 - The == operator returns true if the two reference variables point to the same object; i.e., the two reference variables contain the same address. For example, what does this code fragment print?

```
Car car1 = new Car();
Car car2 = car1;
if (car1 == car2)
{
    System.out.println("the same");
}
else
{
    System.out.println("different");
}
```

Testing Objects for Equality

13

- == operator - compares the reference
- equals method - compares the contents

Testing Objects for Equality

14

- Write an equals method for a Car2 class. Use this skeleton:

```
public class Car2
{
    private String make;
    private int year;
    private String color;
    <equals method goes here>
} // end class Car2

public class Car2Driver
{
    public static void main(String[] args)
    {
        Car2 hamoudCar = new Car2();
        Car2 jessicaCar = new Car2();
        ...
        if (hamoudCar.equals(jessicaCar))
        {
            System.out.println("cars have identical features");
        }
    }
}
```

Passing References as Arguments

17

```
public class PersonDriver
{
    public static void main(String[] args)
    {
        Person person1 = new Person();
        Person person2 = new Person();
        person1.setName("Kamal");
        person2.setName("Luis");

        System.out.println(person1.getName()
            + ", " + person2.getName());

        person1.swapPerson(person2);

        System.out.println(person1.getName()
            + ", " + person2.getName());
    } // end main
} // end PersonDriver

public class Person
{
    private String name;

    public void setName(String name)
    {
        this.name = name;
    }

    public String getName()
    {
        return this.name;
    }

    public void swapPerson(Person otherPerson)
    {
        String temp;
        temp = otherPerson.name;
        otherPerson.name = this.name;
        this.name = temp;
    } // end swapPerson
} // end Person
```

Aside: Swapping algorithm

18

- Write a pseudocode fragment that swaps the contents of the x and y variables. More specifically, fill in the swap code below such that the output is "x=8, y=3".

```
x ← 3
y ← 8
<swap code goes here>
print "x=" + x + ", y=" + y
```

Method-Call Chaining

20

```
johnCar.setMake("Toyota");
johnCar.setYear(2008);
```

- Chaining two method calls together:

```
johnCar.setMake("Toyota").setYear(2008);
```

- That's called *method-call chaining*. It's when you use a dot to concatenate a method call to the end of another method call.

Method-Call Chaining

21

```
public class Car3Driver
{
    public static void main(String[] args)
    {
        Car3 car = new Car3();

        car.setMake("Toyota").setYear(2008).printIt();
    } // end main
} // end class Car3Driver
```

a method-call chain

Method-Call Chaining

22

```
public class Car3
{
    private String make;
    private int year;

    public Car3 setMake(String make)
    {
        this.make = make;
        return this;
    } // end setMake

    public Car3 setYear(int year)
    {
        this.year = year;
        return this;
    } // end setYear

    public void printIt()
    {
        System.out.println(make + ", " + year);
    } // end printIt
} // end class Car3
```

The return type is the same as the class name.

Return the calling object.

Overloaded Methods

24

- Suppose there's a need to perform the same sort of task on different sets of arguments. For example, suppose you want to find the average for these different sets of arguments:
 - two integers
 - three integers
 - two doubles
- One solution is to write three methods with three different names. Here's how you might call those methods:


```
x = findAverageFor2Ints(20, 8);
y = findAverageFor3Ints(5, -3, 18);
z = findAverageFor2Doubles(1.2, 4.0);
```
- What's wrong with that solution?

Overloaded Methods

25

- For the find-the-average example, you could write three overloaded `findAverage` methods and call them like this:

```
x = findAverage(20, 8);
y = findAverage(5, -3, 18);
z = findAverage(1.2, 4.0);
```

Overloaded Methods

26

```
class Height
{
    double height; // a person's height
    String units; // unit of measurement (e.g., cm for centimeters)

    public void setHeight(double height)
    {
        this.height = height;
        this.units = "cm";
    }

    public void setHeight(double height, String units)
    {
        this.height = height;
        this.units = units;
    }

    public void print()
    {
        System.out.println(this.height + " " + this.units);
    }
} // end class Height

public class HeightDriver
{
    public static void main(String[] args)
    {
        Height myHeight = new Height();

        myHeight.setHeight(72.0, "in");
        myHeight.print();
        myHeight.setHeight(180);
        myHeight.print();
    } // end main
} // end class HeightDriver
```

Note that the overloaded `setHeight` methods have different numbers of parameters.

Overloaded Methods

28

- Suppose that you have overloaded methods and you're inside one of the other overloaded methods. Note that it's OK to call one of the other overloaded methods.
- For example, you can replace the original one-parameter `setHeight` method with the following implementation, which calls the two-parameter `setHeight` method.

```
public void setHeight(double height)
{
    setHeight(height, "cm");
}
```

No need for a reference variable dot prefix here.

Constructors

29

- For example, you could define a single `initCar` method to initialize `Car` objects and use it like this::

```
Car brandonCar = new Car();
brandonCar.initCar("Porsche", 2006, "beige");
```

```
Car brandonCar = new Car("Porsche", 2006, "beige");
```

Constructors

30

- A constructor lets you specify what happens to an object when it is instantiated with `new`.
- A constructor is called automatically when an object is instantiated.
- A constructor's name = the object's class name.

Example Car Class with a Constructor

31

```
public class Car4Driver
{
    public static void main(String[] args)
    {
        Car4 chrisCar = new Car4("Prius", 2008, "blue");
        Car4 goldengCar = new Car4("Volt", 2011, "red");

        System.out.println(chrisCar.getMake());
    } // end main
} // end class Car4Driver
```

constructor
calls

Example Car Class with a Constructor

32

```
public class Car4
{
    private String make; // car's make
    private int year;    // car's manufacturing year
    private String color; // car's primary color
```

```
//*****
```

```
public Car4(String m, int y, String c)
{
    this.make = m;
    this.year = y;
    this.color = c;
} // end constructor
```

constructor
definition

```
//*****
public String getMake()
{
    return this.make;
} // end getMake
```

Style requirement:
Put constructors above
a class's methods.

```
} // end class Car4
```

Constructors

33

- Any time you instantiate an object (with `new`), there must be a matching constructor. That is, the number and types of arguments in your constructor call must match the number and types of parameters in a defined constructor.

Will this program compile successfully?

34

```
import java.util.Scanner;

public class Employee
{
    private String name;

    public void readName()
    {
        Scanner stdIn = new Scanner(System.in);
        System.out.print("Name: ");
        this.name = stdIn.nextLine();
    } // end readName
} // end class Employee

public class EmployeeDriver
{
    public static void main(String[] args)
    {
        Employee emp = new Employee();
        emp.readName();
    } // end main
} // end class EmployeeDriver
```

Will this program compile successfully?

35

```
import java.util.Scanner;

public class Employee2
{
    private String name;

    public Employee2(String n)
    {
        this.name = n;
    } // end constructor

    public void readName()
    {
        Scanner stdIn = new Scanner(System.in);
        System.out.print("Name: ");
        this.name = stdIn.nextLine();
    } // end readName
} // end class Employee2

public class Employee2Driver
{
    public static void main(String[] args)
    {
        Employee2 waiter = new Employee2("Derrick");
        Employee2 hostess = new Employee2();

        hostess.readName();
    } // end main
} // end class Employee2Driver
```

Overloaded Constructors

37

```
public class Fraction
{
    private int numerator;
    private int denominator;
    private double quotient;

    public Fraction(int n)
    {
        this(n, 1);
    }

    public Fraction(int n, int d)
    {
        this.numerator = n;
        this.denominator = d;
        this.quotient =
            (double) this.numerator
            / this.denominator;
    }

    public void printIt()
    {
        System.out.println(
            this.numerator +
            " / " + this.denominator +
            " = " + this.quotient;
        ) // end printIt
    } // end Fraction class

    public class FractionDriver
    {
        public static void main(String[] args)
        {
            Fraction a = new Fraction(3, 4);
            Fraction b = new Fraction(3);
            // Fraction c = new Fraction(); // error
            a.printIt();
            b.printIt();
        } // end main
    } // end class FractionDriver
```

Constructor chaining

29

- Constructor chaining is the process of calling one constructor from another constructor with respect to current object.
- Main use: avoid duplicate codes while having multiple constructor (by means of constructor overloading)
- Two ways: within same class from base class

Classes with Class Members

1

- Class Variables
- Class Methods
- How to Access Class Members
- When to Use Class Members
- Class Constants
- Example Program Using Class Members

Class Variables

3

- If you'd like a variable to be shared by all the objects within the variable's class, make it a class variable by using the `static` modifier in its declaration:

```
<private or public> static <type> <variable-name>;
```

- Example:

```
public class Mouse
{
    private static int mouseCount;
    private static double averageLifeSpan;
    ...
}
```

- Class variables are declared at the top of the class, above all the methods.

Class Variables

4

- Class variables use the same default values as instance variables:
 - integer types get 0
 - floating point types get 0.0
 - boolean types get false
 - reference types get null
- What are the default values for the class variables in this code fragment?

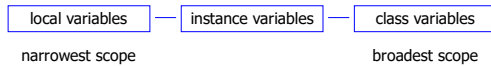
```
public class Mouse
{
    private static int mouseCount;
    private static double averageLifeSpan;
    private static String researcher;
    private static int simulationDuration = 730;
    ...
}
```

Initializations
are allowed.

Scope

6

- Scope continuum:



- Narrower scope equates to more encapsulation, and encapsulation means you are less vulnerable to inappropriate changes.
- Class variables, with their broad scope and lack of encapsulation, can be accessed and updated from many different places, and that makes programs hard to understand and debug.

Class Methods

7

- If you have a method that accesses class variables and not instance variables, then you should declare the method to be a *class method*. To do so, add `static` to the method's heading like this:

`<private-or-public> static <return-type> <method-name>(<parameters>)`

- Example:

```

public class Mouse
{
    private static int mouseCount;
    private static double averageLifeSpan;

    public static void printMouseCount()
    {
        System.out.println("Total mice = " +
            Mouse.mouseCount);
    }
}
  
```

To access a class variable, prefix it with `<class-name> dot`.

How to Access Class Members

8

- Normally, to access a class member (a class variable or a class method), prefix it with `<class-name> dot`.
- Shortcut syntax for a class member:
 - In accessing a class member, you may omit the `<class-name> dot` prefix if the class member is in the same class as where you're trying to access it from. For example:

```

public class Mouse
{
    private static int mouseCount;
    private static void printMouseCount()
    {
        System.out.println("Total mice = " + Mouse.mouseCount);
    }
    public static void main(String[] args)
    {
        Mouse.printMouseCount();
    }
} //end Mouse class
  
```

OK to access `printMouseCount` without `<class-name> dot`.

OK to access `mouseCount` without `<class-name> dot`.

Class Constants

10

- To make a class constant, declare a variable with the `static` and `final` modifiers like this:


```
<private or public> static final <type> <variable-name> = <initial value>;
```
- Note that class constants should be assigned as part of an initialization statement. If you attempt to assign a value to a class constant later on, that generates a compilation error.
- As with most variables, class constants usually use the `private` modifier.

Class Constants

11

- In the Human class below, we make `NORMAL_TEMP` a class constant (with the static and final modifiers) because all Human objects have the same normal temperature of 37° Celsius.

```
public class Human
{
    private static final double NORMAL_TEMP = 98.6;
    ...
    public boolean isHealthy()
    {
        return Math.abs(currentTemp - NORMAL_TEMP) < 1;
    } // end isHealthy

    public void diagnose()
    {
        if ((currentTemp - NORMAL_TEMP) > 5)
        {
            System.out.println("Go to the emergency room now!");
            ...
        }
    } // end class Human
```

class constant